Learning Log — Bronze Layer Ingestion

pyspark-lakehouse-aws · Stage 1 (raw ingestion to bronze) · 2026-08-04

What got built

- `src/bronze/ingest_dimensions.py` — generic ingestion function: reads a raw CSV by name with an explicit schema, writes it to the MinIO `bronze` bucket. Driven by a loop over all five Olist dimension tables.
- `config/schemas.py` — one `StructType` per dimension table (customers, products, sellers, geolocation, category translation), collected in a `SCHEMAS` lookup dict keyed by file name.
- `config/spark_session.py` — cleaned up: removed the now-dead `spark.jars.packages` config, since package resolution moved to the `spark-submit` command line.
- `docker-compose.yml` — added `PYTHONPATH` and `PATH` to the `spark` service so `spark-submit` and cross-package imports (`config` from `src`) work from any directory.

Concepts learned

Schema-on-read vs. self-describing formats SPARK

CSV carries no type information — Spark has to either infer it (a real risk) or be told explicitly. Parquet/Delta/Iceberg embed their schema in file/table metadata, so once data is written, downstream reads need no schema declaration at all. This means schema effort concentrates at exactly one point: the raw → bronze read.

inferSchema is not a safe default BUG FOUND

Concrete example from this dataset: `customer_zip_code_prefix` values like `"01310"` get inferred as `IntegerType` , silently dropping the leading zero and becoming `1310` . No error, no warning — just quietly wrong data. Same risk applied to `seller_zip_code_prefix` and `geolocation_zip_code_prefix` . Fix: explicit `StructType` with `StringType()` for any ID/code-like column, decided deliberately rather than inferred.

`StructType` vs. DDL string schema

`StructType` / `StructField` objects support a `metadata` dict (useful for column descriptions, e.g. feeding Glue Data Catalog comments later) and are directly comparable in tests (`df.schema == expected`). DDL strings (`"col STRING NOT NULL"`) are more compact but only parsed/validated at runtime. Nullability constraints in either form are **not actually enforced** by Spark on file-source reads (CSV especially) — they're documentation/intent, not a runtime guarantee. Landed on: `nullable=True` everywhere at bronze, since bronze's job is to preserve raw data as-is; real null-handling belongs to the cleaning stage.

Python's `sys.path` resolution differs by invocation DEBUGGING

`python3 src/bronze/ingest_dimensions.py` only adds the *script's own directory* to `sys.path` — not the working directory. Since `config/` is a sibling of `src/`, not nested inside it, `from config.spark_session import ...` failed with `ModuleNotFoundError` even though the file existed. `python3 -m src.bronze.ingest_dimensions` adds the current working directory instead, which fixes it. Longer-term fix: set `PYTHONPATH` in the environment so plain script invocation works too.

spark-submit changes when package resolution happens PRODUCTION-RELEVANT

Running `python3 script.py` directly works "by accident" for `spark.jars.packages` set in code — PySpark's gateway launcher reads that config and internally shells out to its own `spark-submit --packages ...` before the JVM starts. Running `spark-submit` yourself skips that: the JVM (and its classpath) is already fixed by the time your Python code's `.config("spark.jars.packages", ...)` runs, so it's a no-op — resulting in `ClassNotFoundException: S3AFileSystem`. Fix: declare packages on the `spark-submit` command line itself (`--packages org.apache.hadoop:hadoop-aws:3.3.4,...`), which also matches how real deployments manage dependencies — as a submit-time concern, not baked into application code.

How this generalizes to a real cluster

`--master` targets a real cluster manager (YARN / `k8s://...` / standalone) instead of `local[*]`. `--deploy-mode cluster` runs the driver inside the cluster itself rather than on the submitting machine, so the job survives a disconnect. Most importantly: `PYTHONPATH` only works because everything runs on one machine locally — on a real cluster, executors are separate processes on separate machines with no shared filesystem, so your own code (like `config/`) has to be explicitly shipped via `--py-files` (zip/wheel) or baked into the container image (Kubernetes). Credentials also change: static `fs.s3a.access.key` / `secret.key` (fine for local MinIO) would be replaced by an IAM role (instance profile / IRSA) in real AWS.

Python environment hygiene TOOLING

Homebrew's Python is "externally managed" (PEP 668) — `pip install` at the system level is blocked on purpose to avoid breaking the Homebrew installation. Correct fix is a project-scoped venv (`python3 -m venv .venv`), not `--break-system-packages`. Also clarified: the local venv's `pyspark` is *only* for editor import resolution (Pylance/autocomplete) — it's never actually executed. The real job always runs inside the Docker container, which has its own independent PySpark install. The two never interact.

Open item for next session

- Write a test asserting `customer_zip_code_prefix` (and the other zip-prefix columns) load as strings with leading zeros intact — this is the one thing standing between today's work and calling stage 1 fully done per the repo's own testing bar.

- Minor cleanup: remove commented-out dead code (`show(5)`), stray trailing comma, and finish the half-written comment in `ingest_dimensions.py` .
